The Latent Monitor Audit: What We Built, How It Works, and What It Found

A technical report on the inference-only study of continuous-thought reasoners

Muhammad Jon Raza
Project repository: `0x0shephard/latent-reasoning`
Research ledger §108–§136, October 2026

Abstract

Some language models have been trained to reason in six hidden vectors ("thoughts") instead of written steps. Several groups propose to monitor such models by training probes on those vectors. This report explains, for the team, what we did to test whether that is sound. We built an inference-only audit that separates three questions a probe conflates: does a thought *carry* a value into the answer, does the model *need* the thought, and is what a probe *reads* from the thought confirmed by behaviour. We ran it on eight released checkpoints (CODI on GPT-2 and LLaMA-1B, SIM-CoT at 1B, 3B and 8B, a Coconut retrain, and Coconut and CODI on the ProsQA logic task), five datasets and 39 preregistered Kaggle runs that reproduce byte for byte across sessions. On the CODI family, readable is not used and used is not needed: below 3B a thought carries a value only on problems of three or more steps, one or two thoughts do the carrying and hand the value to the next position's memory, the last thought is readable and never used, the carriers can be removed at little cost, probes read the question from every thought, and the thoughts predict wrong answers no better than the model's own confidence. The one practical gain is not in the thoughts: routing the least-confident quarter of problems to the model's own written chain gains 3.7, 7.5 and 10.2 points at 1B, 3B and 8B. This report walks through the motivation, each instrument with a worked example, every finding with the experiment behind it, what is new against the literature, and the first result of the preregistered next experiment: a targeted latent hijack. On CODI-1B one natural vector written into the carrier thought moves 21% of correct answers to an attacker-chosen value (control 1.5%), the same write at the readable-but-unused slot sets off a monitor on 100% of victims and changes nothing, and, against our prediction, the downstream slots keep the original value, so a detector on the slot after the carrier catches the hijack at 0.92 AUROC.

1 Why we did this

The models. CODI (Shen et al., 2025), SIM-CoT (Wei et al., 2026) and Coconut (Hao et al., 2024) are trained to solve grade-school math word problems without writing their steps. After the question the model produces a fixed number of continuous vectors, six in every checkpoint we study, and then the answer. Each thought is one forward pass: its last hidden state is fed through a small projector as the input of the next thought, and like any token position it also writes keys and values into the attention cache that every later position can read. Nobody can read the thoughts directly.

Preprint. Under review.

The proposal we tested. A natural response is to read the thoughts with a probe, a small classifier trained after the fact on recorded vectors. Ramjee et al. (2026) trained a backdoored Coconut model and showed a probe on the early thoughts detects the "armed" state at 89%, and recommended monitoring the early latent positions. That result, like every latent-probe result we could find, is correlational: nothing in it shows the probed vector is used by anything downstream. A probe can read a note the recipient threw away.

The contradiction that started the chapter. We had spent three months on this model family with a different goal (making CODI more efficient), and had built verified tooling to swap, delete and overwrite individual thoughts in the official checkpoints. Two facts sat uneasily next to each other. Our own interchange experiments showed that swapping the *state* of thought 1 or 3 between problems changed about a third of the answers. SCIT (Ding et al., 2026), on the same checkpoint, swapped the *keys and values* and reported that the value cache at layers 8–9 carries the answer while the hidden state "is weak". Both can be true only if a thought acts through two separable routes, and a linear monitor reads just one of them. If the computation travels through the cache, a probe on the state reads the wrong object. And one architectural fact made this sharp: the released CODI code discards the sixth thought's projected output. A probe on z_6 reads a vector that goes nowhere.

The question. When a thought affects the answer, which route carries the effect, does it carry a *value* or merely an influence, does the model need it, and does a probe's reading track any of this? Every one of these can be answered from released weights, with no training, in about one GPU-hour per model. That is what we built.

2 Models and data

Table 1: Checkpoints. Accuracies are our greedy GSM8K-test reproductions on 1,319 problems. The CODI-1B figure corrects a number quoted by later papers (51.9% is an ablation row of the CODI paper; the main result is 55.6%).

Name	Base	Source	Paper	Ours
CODI GPT-2	GPT-2 (124M)	`zen-E/CODI-gpt2`	43.7	43.6
CODI-1B	LLaMA-3.2-1B-Instruct	`zen-E/CODI-llama3.2-1b-Instruct`	55.6	55.5
SIM-CoT-1B	LLaMA-3.2-1B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-1B`	56.1	55.7
SIM-CoT-3B	LLaMA-3.2-3B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-3B`	62.3	61.9
SIM-CoT-8B	LLaMA-3.1-8B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-8B`	64.1	62.5
Coconut	GPT-2 (124M)	`UdeSA/coconut-0.2B-gsm8k-aug` (retrain)	34.1	34.6
Coconut on ProsQA	LLaMA-3.2-1B-Instruct	`connordilgren/llama32-1b-prosqa-coconut`	97.6	97.6
CODI on ProsQA	LLaMA-3.2-1B-Instruct	`connordilgren/llama32-1b-prosqa-codi`	99.0	99.0

Every experiment needs problems whose solution is an equation chain, written as `«8+1=9» «9*80=720» «720-700=20»`, because the twin construction recomputes the chain. Pools: the GSM8k-Aug training split CODI was trained on (2,048 rows, plus 14,336 extra rows for long chains), the original GSM8K test set with its calculator annotations as the chain (1,206 usable rows, 854 with three or more steps), the original GSM8K train set (6,998 rows), ASDiv (1,490 rows, only 86 with three or more steps), and SVAMP as a one-step control. "Short" means at most two steps; "long" means three or more. All runs executed on Kaggle T4 GPUs through the API, from two accounts.

3 The instruments, explained

Everything below is an intervention: we change something inside a forward pass and watch the answer. Feeding a thought its own recorded state or K/V reproduces the original logits to the last bit; that identity test runs before every measurement and is the check that the plumbing is right.

3.1 Two routes out of every thought

A thought at slot s affects the answer in two ways. Its **state** is the vector it hands to slot $s{+}1$ through the projector (the front door). Its **K/V** are the keys and values it writes at its own cache position, which every later position attends to (the back door). Our hooks edit either alone: replace the state

before it is projected, or overwrite the keys and/or values at that position, on any subset of layers, after they are written.

One fact follows from the architecture and we verified it exactly on all eight checkpoints: replacing all six states *and* all six positions' K/V is identical to replacing the K/V alone. The last state is discarded, so everything a state does, it does through the K/V its successors write. This identity is the reason a monitor on states is reading an *intermediate* object.

Worked example: the two-route audit on CODI-GPT2 (§108–§109)

512 training problems, each paired with an unrelated problem. At each slot we swap in the other problem's state only, K/V only, values at SCIT's layer groups, or the whole position, and count how often the first answer token changes. Result: the state route is strong at the odd slots (0.31, 0.37) and weak at the even ones; the K/V route is strong at the even positions (0.42, 0.24, 0.28) and weak at the odd ones; within K/V the carrier is the values at layers 8–9. The sixth thought moves 0% through its state and 3.9% through its K/V. A complete tail transplant changes 75% of answers, but only 4.7% become the donor's answer: the readout fuses the stored values with the recipient's question.

3.2 Twins: does a thought carry a value, or only disturb?

Swapping a thought from an unrelated problem breaks answers. Breaking is not carrying. To test carrying we build **twins**: change one integer in the question and recompute the chain, so the twin's answer is the correct answer to the changed question. "12 cucumbers" becomes "13 cucumbers" and `«12*5=60» «54+60=114»` becomes `«13*5=65» «54+65=119»`. A number is eligible only if it appears exactly once in the question and once as an operand, is not itself a result, and the recomputed results stay non-negative integers; the smallest $\delta \in \{\pm1, \pm2, \pm3, \pm5, \pm10, \pm20\}$ that works is used.

- **Competence gate.** We keep a pair only if the model solves both twins natively and at the first token, and their gold first tokens differ. Otherwise "the answer followed the swapped value" has no meaning.
- **Intervention.** The twins sit adjacent in one batch so position ids match; each is the other's donor; both directions are pooled. We replace one site in the recipient (a thought's state, a position's K/V, or whole tails) with the twin's and read the first answer token.
- **Outcome.** *Target* if the token becomes the twin's answer, *retain* if it stays the recipient's, *other* otherwise. The target rate is **transport**. Intervals come from a paired bootstrap over pairs (2,000 resamples).
- **Profiles.** *Standard*: perturb the first eligible number; mostly short chains. *Long*: only chains of three or more steps, perturb the number that enters the chain latest. Both cap at 512 pairs.
- **Locating the value.** The model's own output head, applied to a thought's state, decodes a few numeric tokens. If the recipient decodes its own changed value at some slot and the twin decodes the twin's at the same slot, the value is *located* there.

3.3 Masking: does the model need the thought?

Sufficiency is not necessity. For each thought we run two removals: *mean* replaces its state by the pool-mean state, so nothing problem-specific reaches the next thought; *mask* hides its K/V from every later position through the attention mask (this needs a decoding path that carries the mask and explicit position ids through every step; the released LLaMA path drops the mask, and we fixed that). We report the accuracy drop in points, split by chain length, with a bootstrap over rows. Three preregistered checks: short chains survive any single removal within 5 points (K1), carriers cost at least 10 points on long chains (K2), the last thought costs at most 2 points (K3).

3.4 Probes: readable, and faithful?

A probe is a small model trained after the fact to predict a label from a recorded thought vector. We use three readers: the model's own output head (no training), a ridge probe to the embedding of the target number token decoded by nearest numeric token (linear), and a two-layer MLP classifier over roughly 900 numeric tokens (non-linear). The target is the chain's penultimate intermediate result,

the value the carriers hold (the final answer itself is unreadable from any thought, at 0.03). Folds are assigned by twin pair so a probe is never read on a problem whose twin it trained on. **Readability** is held-out accuracy.

Faithfulness is the part that makes this an audit rather than a probing study. After a swap at thought s, the recipient holds the twin's state there. A monitor placed at s would read the twin's value v' and raise the alarm "the model is working with v' now". Faithfulness is the share of those alarms the behaviour confirms: how often the answer actually became the twin's.

$$\text{faithfulness}(s) \;=\; \frac{\#\{\text{rows: probe reads } v' \text{ at } s \;\wedge\; \text{outcome} = \text{target}\}}{\#\text{rows}}.$$

A thought can be readable and never faithful. The same machinery with different labels gives the **prompt-leakage probes** (the chain's first operator; whether the problem has three or more steps), properties the question fixes before any reasoning.

3.5 The card

One driver runs the twins (both profiles), the masking audit, both probe families and the bootstraps, and writes one card per (model, dataset, GPU session). Fixed thresholds: a *carrier* has long-chain state transport ≥ 0.15; a *store* has K/V transport ≥ 0.15 at a position whose state does not carry; the short-chain ceiling is 0.10 (position 0's K/V excepted, since it holds the question); *necessary* means ≥ 10 points on long chains; *readable* is probe accuracy ≥ 0.30; *readable-but-unfaithful* (C4b) is readable with an alarm confirmed $\leq 5\%$ of the time. Five checks (C1 no transport on short chains; C2 a carrier exists on long chains; C3 the last thought is inert, unfaithful and unnecessary; C4 readability dissociates from faithfulness; C5 every carrier is necessary) give a *regime*: `critical_path` if C5 holds, `backup_route` if carriers exist but C5 fails, `no_transport` if there is no carrier. The card records GPU, library versions, dataset hash and a SHA-256 fingerprint of every twin run's predictions; a comparison tool flags disagreements across pools and sessions.

4 Findings on the CODI family

Each finding states the claim, how we got it, and the numbers. Section numbers refer to the research ledger.

F1. Disturbing a thought is not the same as the thought carrying a value (§109, §111)

Under an unrelated donor, swapping thought 3's state changes 37% of answers; under a twin that differs in one number, the same swap moves the answer to the correct counterfactual 10% of the time and leaves the own answer in place 86–92% of the time. Thought 1: 31% vs 5%. The 31–37% was disruption. *How:* the same 512-problem pool scored two ways, random-donor change rate and twin target rate.

F2. Below 3B, a thought carries a value only on problems with three or more steps (§111–§113, §117)

On one- and two-step problems every single-site transport is ≤ 0.10; the readout computes the answer from the question and the thoughts are a readable but causally redundant trace. On chains of three or more steps, thought 3's state transports the value in a third of rows: 0.325 [0.293, 0.358] at GPT-2, 0.339 [0.304, 0.375] at CODI-1B. *How:* the long profile (14,336 extra training rows to reach 512 gated pairs, the latest-entering number perturbed), preregistered after the stratum effect appeared in cells of 36–38 rows. The card codifies it as C1 and C2.

F3. One or two thoughts carry; they hand the value to the next position's memory; the last thought (below 3B) never matters (§109, §113, §117, §120)

The carried value travels from the carrier's state into the *whole* K/V of the following position (CODI-1B: state_3 0.339, K/V_4 0.260, state_4 0.000, K/V_3 0.000), not into SCIT's two-layer value block (0.047 at GPT-2): the value block is what *breaks* answers, not what *carries* them. Where the model's own readout localises the value (CODI-1B, 60% of rows uniquely at slot 3), the located thought carries 0.529 against 0.000 at the other feeding thought: the thought that decodes the value is the thought that carries it. The sixth thought decodes the value as well as thought 3 and transports 0.000 through both routes on 590 rows; masking it costs ≤ 1.9 points on every model up to 1B.

F4. The carrier is used but barely needed; the question register is the most necessary site (§120–§121, §124)

At CODI-1B the carrier transports a third of the time with total specificity, and mean-ablating it costs 2.7 points (K2 fails outright); hiding its K/V costs 0.0. The model recomputes from the question when a thought is removed: *sufficiency without necessity*, the `backup_route` regime, which turned out to be the common case on natural GSM8K questions. Meanwhile hiding position 0's K/V, a copy of the question written before any thought, costs 37.6 points at GPT-2 and 58.6 at SIM-CoT-1B on long chains. A monitor that reads thoughts and ignores position 0 misses the site the model leans on most.

Table 2: Monitor readability / faithfulness on long chains (ridge probe, penultimate intermediate). In every model the terminal slot is readable and faithful at exactly 0.000.

slot	GPT-2	CODI-1B	SIM-CoT-1B
0	0.09 / 0.00	0.27 / 0.02	0.25 / 0.02
1	0.33 / **0.19**	0.15 / 0.01	0.62 / **0.21**
2	0.46 / 0.00	0.36 / 0.02	0.69 / 0.02
3	0.40 / **0.40**	0.79 / **0.36**	0.72 / **0.28**
4	0.54 / 0.00	0.84 / 0.05	0.74 / 0.03
5	0.42 / **0.00**	0.81 / **0.00**	0.57 / **0.00**

F5. Readable is not faithful; probes read the prompt; the readable-unfaithful slot is a stale copy (§120–§121, §124–§125, §133)

Table 2: CODI-1B thought 4 reads at 0.84 and its alarm is confirmed 5% of the time; thought 5 reads at 0.81 and is confirmed never. Faithfulness changes by less than 0.05 between the ridge and MLP probe families, so it is a property of the slot, not the probe. A stronger statement we first made, "the most readable slot is inert", was withdrawn when the MLP probe put the carrier on top in some cards; what survives both families on every card up to 3B is C4b: a slot read at ≥ 0.30 whose alarm is confirmed $\leq 5\%$, and the last thought is that slot in every model. Prompt-leakage probes decode "is this a $\geq$ 3-step problem" at 0.76–0.95 from *every* slot, position 0 and the unused last thought included, and the best slot for the first operator is never a carrier. Finally, the readable-unfaithful slot is not a prompt reader: on CODI-1B slot 5 reads the computed penultimate value at 0.47, *above* the carrier's 0.42. It holds a copy of the value the carrier already handed to the cache. Readability outlives use.

F6. The carrier pairing is trained, not architectural; step supervision moves it as predicted (§115, §118–§119)

GPT-2: pairs (1→2) and (3→4); CODI-1B: (3→4) alone; SIM-CoT-1B, trained with per-step supervision on the same backbone: (1→2) and (3→4) both active (0.191/0.185, 0.247/0.229 against CODI-1B's 0.011/0.011 and 0.339/0.260), the prediction written before the run. SIM-CoT also turns the first latent position into a question register that transports 0.54 of short-chain twins through its K/V alone. What did not hold: we predicted step supervision would make the free readout slot-specific; unique location fell from 60% to 6%, because the model's tied output head is not the decoder SIM-CoT trained.

5 Four uses of the map

5.1 A monitor can read the prompt instead of the plan (§125)

Properties fixed by the question (first operator, four classes; ≥ 3 steps, binary) decode above chance at every slot of every model, including the two that do nothing. A monitor evaluated on a prompt-borne property (a trigger word, a task type, a difficulty) can score well while reading nothing the model computed. For a trigger-in-the-prompt backdoor evaluation this is the confound, and the fix is cheap: run the same probe on a prompt-only control at the same slot.

5.2 Thoughts do not catch errors better than confidence (§125, §131, §134)

For every GSM8K test problem we recorded the thoughts and the answer, applied the model's own output head to each thought (top-1 value, probability, entropy, agreement with the answer and other slots), and trained five-fold logistic classifiers to predict "the answer is wrong": carriers only, inert only, all slots, raw states, and the answer's own first-token confidence (Table 3). Confidence wins on every model; the thoughts add ≤ 0.01 AUROC to it, even with 12,288 raw features. But on the 1B–8B models the error signal inside the thoughts sits exactly at the carriers the swaps found (CODI-1B carriers 0.82 vs inert 0.60), an independent confirmation of placement from a task the audit was never fitted to.

Table 3: AUROC for "the native answer is wrong" on the full GSM8K test set (1,319 rows).

Model	Wrong	Confidence	Carriers	Inert	All slots	Raw states
GPT-2	746	**0.889**	0.850	0.853	0.851	0.859
CODI-1B	587	**0.912**	0.821	0.596	0.827	0.835
SIM-CoT-1B	584	**0.913**	0.754	0.650	0.799	0.836
SIM-CoT-3B	502	**0.915**	0.793	0.688	0.807	0.844
SIM-CoT-8B	495	**0.930**	0.763	0.665	0.794	0.824

5.3 A value-matched write into the carrier (§126–§130)

The audit says the carrier holds the intermediate value and passes it on. Repair asks whether the channel works in reverse.

Worked example (a real CODI-1B case)

"The elevator in Jack's building supports a maximum load of 700 kg. An adult weighs on average 80 kg. If Jack rides with 8 other adults, by how much will they exceed the maximum load?" Gold chain `«8+1=9» «9*80=720» «720-700=20»`, so $v^* = 720$. CODI-1B answers 40; its output head reads 640 at the carrier: it forgot Jack. In a bank of 2,048 solved training problems the carrier reads 720 at five of them. We copy the most similar one's carrier vector in at thought 3 while decoding, and read the new answer: 20. Repaired. With a random different-value donor the answer stays 40.

Protocol. Bank: 2,048 training rows decoded once, states and K/V recorded, indexed by the value the model's own head reads at the carrier. Targets: wrongly answered $\geq$ 3-step test problems (cap 512) plus 128 correctly answered controls for harm. Conditions: nearest same-value donor (cosine), random same-value, the *centroid* of same-value states (a synthetic vector), *random different-value* (the key control: if it repaired as often, the repair is disturbance), store K/V, state plus K/V, the whole donor tail. Rates with bootstrap over targets on the primary stratum (wrong targets whose carrier reads something other than v^* and that have a donor).

What we learned. The carrier is a writable channel: 14–23% repair against a 2–4% different-value control up to 3B, with non-overlapping intervals and 5–9% harm at 1B/3B. Two predictions were wrong in useful ways: the centroid repairs as well as a retrieved natural vector (retrieval buys nothing over averaging), and adding the store's K/V *lowers* repair (the donor's memory carries the donor's problem). A learned writer from a value's token embedding to a carrier vector (§128) lands in the same 20–24% band; writing several step-mapped slots at once (§129) is level or worse and harms more. With the value from the model's own explicit chain plus a calculator, repair survives at

Table 4: Repair rate (%) on the primary stratum; harm (%) on correct controls. Gold-chain value.

Model	n	Nearest donor	Random same	Centroid	Different value	Store K/V	Harm
GPT-2	357	15.1 [11.5, 19.0]	17.6	20.2	3.6	11.5	18.7
CODI-1B	266	22.6 [17.7, 27.8]	21.8	23.7	2.6	10.2	7.0
SIM-CoT-1B	273	15.4 [11.4, 19.8]	9.9	13.2	2.6	4.0	5.2
SIM-CoT-3B	239	14.2 [10.0, 18.8]	12.1	15.9	1.7	6.3	8.6
SIM-CoT-8B	232	14.7 [10.3, 19.4]	14.7	13.4	6.0	5.2	7.8

CODI-1B (18.8%) but the chain's own answer is right on more of the same failures than injection recovers.

Why the ceiling is low: the failure taxonomy (§129–§130, §133). Classing every wrong answer by what the model's own written chain does on that problem: *plan wrong* 48–68% (the written chain is wrong too), *latent-only* 19–53% (right when writing, wrong when thinking silently), *upstream arithmetic* 6–11%, *final step* 1–9%. Writes reach $\leq 2.6\%$ of plan-wrong failures; the one-slot write already repairs a third of touched latent-only failures. Latent-only failures are hard problems, not slips: a peer model of the same size misses them 60% of the time, and failure sets overlap at Jaccard 0.70 against 0.34 under independence. Handing the model the gold plan at inference (§132) recovers 6–8% of plan-wrong failures against 3–7% for a *wrong* plan: disturbance, not content.

Net accuracy. On the 838 long-chain test problems the best gated thought write gains -0.1 / $+3.4$ / $+1.1$ / $+0.4$ / $+3.4$ points (GPT-2 / CODI-1B / SIM-CoT-1B / 3B / 8B), roughly $0.64\times$ that on the full set. Switching to the explicit chain on disagreement gains $+5.6$ / $+5.3$ / $+3.2$ / $+9.7$ / $+21.3$. The write is a mechanism result, confirmed by two independent sources of the value, not a method.

5.4 Route on confidence: the cheap gain (§131, §133–§134)

Confidence is the best error detector available and the model's written mode is right on many latent failures, so we used confidence as a router: run the latent pass on everything, send a problem to its own explicit chain only when the first-token probability is below a threshold, sweep the threshold (Table 5). No new GPU time for four models; the data were already on disk.

Table 5: Confidence-gated cascade: net accuracy change (points) on the 838 long-chain GSM8K test problems when the least-confident fraction is routed to the model's own explicit chain. 95% row-bootstrap intervals at 25% and 50%.

Routed	GPT-2	CODI-1B	SIM-CoT-1B	3B	8B
10%	$+0.8$	$+0.8$	$+0.5$	$+2.7$	$+4.9$
25%	$+1.5$ [-0.2, $+3.2$]	$+3.7$ [$+1.9$, $+5.1$]	$+1.9$ [-0.3, $+4.3$]	$+7.5$ [$+5.1$, $+9.9$]	$+10.2$ [$+8.1$, $+12.6$]
50%	$+3.1$ [$+0.1$, $+5.7$]	$+7.6$ [$+4.5$, $+9.7$]	$+4.9$ [$+1.6$, $+7.8$]	$+11.1$ [$+7.5$, $+14.3$]	$+19.1$ [$+15.9$, $+22.0$]
100% (always explicit)	$+5.6$	$+5.3$	$+3.2$	$+9.7$	$+21.3$
Oracle (route exactly the wrong)	$+12.0$	$+11.5$	$+10.3$	$+17.2$	$+23.9$
Best gated thought write	$+1.0$	$+3.4$	$+1.1$	$+0.9$	$+3.4$

Three things to notice. At 1B and 3B routing *half* beats always writing the chain (7.6 vs 5.3; 11.1 vs 9.7), because the chain is wrong on some problems the confident latent pass had right and the gate keeps it away from them. Adding the six thought states to the gate, as out-of-fold ridge scores, makes it *worse* at 25% routed on three of four models: confidence is the whole gate. And the same confidence partitions the thought write's successes from its harms (it takes on confident failures, 73% vs 38% by quartile at CODI-1B, and breaks loosely held correct answers, harm 19–31% in the bottom quartile and 0–3% in the top), so below the threshold switching wins and above it doing nothing wins; the write beats both in exactly one cell (CODI-1B, high confidence, $+1.2$ vs $+0.5$). **Route on confidence, do not write.**

6 Contrast and scale

Scale (§130, §134). The two findings that defined the small checkpoints are properties of small checkpoints. At 3B the carriers move to the even slots 2 and 4 with the store at 3, slot 4 transports 0.23 on short chains (no gate), and the terminal K/V transports 0.15 and costs 8 points (not inert). At

8B there are two carrier *pairs* specialised by chain length (4→5 for short chains at 0.51, 2→3 for long at 0.28), the question register costs 5 points instead of 33–59, the most necessary site is a thought for the first time, and there is no readable-but-unfaithful slot: readability tracks faithfulness at Spearman 0.71. What survives every scale: the carriers are sufficient but not necessary (`backup_route` at 1B, 3B and 8B), the computation lives at one or two trained slots, the thoughts add nothing to error detection, and the latent–explicit gap grows (5, 10, 21 points).

Coconut is the opposite case (§127). A third-party retrain with the official recipe (34.6% = model card), audited with the same hooks and native outcomes. Its pairing is (0,1), (2,3), (4,5), exactly as the two-thoughts-per-step curriculum predicts, and it held to the third decimal (state 2 ↔ K/V 3 both 0.52). But its thoughts carry on one-step problems (0.45 at the first thought), every pair is necessary (45–49 points), the last position is in use (K/V 0.15, 14 points), and a transplanted thought carries the donor's whole problem (repair 5.6% vs 3.1% control with 29% harm). Same six slots, same task, opposite monitoring profile. The CODI results are properties of CODI's distillation.

ProsQA: nothing monitorable (§135). Coconut and CODI at LLaMA-1B on a synthetic logic task (97.6% and 99.0%, both reproducing their paper). Twins rewrite the path's last hop, checked by graph reachability. Transport is 0.000 at every site and for the whole tail; masking all six thoughts costs 0.000. And rewriting one premise so that the other option is correct is answered with the *old* option 68–79% of the time: the accuracy does not come from following the annotated path. The audit's third profile, `no_transport`: there is nothing in those thoughts to monitor and nothing in them to repair.

7 Preregistration and reproducibility

Every experiment was written into the ledger with its predictions before the run and scored afterwards. Eleven predictions were partly or wholly wrong and are reported as such; the most useful wrong ones were that the 1B readout would not localise (it did, at 60%), that step supervision would make the readout slot-specific (it did not), that the centroid would lose to retrieval (it did not), that the store's K/V would help repair (they hurt), and that the 3B model would have a single carrier at slot 3 (it has two at the even slots). Two errata against our own earlier readings are recorded: a benchmark figure (we had the CODI paper's ablation row as its main result) and a check reported as holding without reading the committed card (C4's correlation half). Five reruns, on two Kaggle accounts and days apart, reproduced their twin-run prediction fingerprints byte for byte. Code, cards, per-target outcomes and the ledger are in the repository; every notebook pins the commit it runs.

8 What is new, against the literature

Checked claim by claim on 2026-10-12 (ledger §122) and for the cascade on 2026-10-17.

ThinkRouter is the one item that must be read in full before submission; if it routes per problem on CODI-style models with first-token confidence, our cascade contribution narrows to the negative half (the thoughts add nothing) and the scale and oracle analysis.

9 Preregistered and running: Build 7, the targeted latent hijack (§136)

The repair line wrote the *right* value into a *failed* problem. The same channel, used by an adversary, writes a *wrong* value into a *solved* problem. This turns the dissociation into an attack–defence pair: if the carrier is writable, downstream slots are copies, and confidence rather than the thoughts carries the error signal, then a one-vector write should steer the answer to a chosen value, every monitor downstream should agree with the attacker, a write at the decoy slot should set off a monitor while changing nothing, and only recomputation should catch it.

Protocol. Victims: up to 512 correctly answered $\geq$ 3-step GSM8K test problems. For each, an attacker value v_{att} is fixed before any write: the bank value nearest in magnitude to v^* whose *implied answer* (the gold chain's last equation re-evaluated with v_{att} in place of its penultimate operand) is a non-negative integer other than the gold answer. Conditions: an own-value control (a different

Table 6: Novelty by finding. "Prior" names the closest work; "ours" is what was not found anywhere.

Finding	Closest prior	Ours
Influence $\neq$ transport; the chain-length gate	Cywiński et al. (2025) on 2-step templates, naming more steps as open; Ding et al. (2026); Li et al. (2026)	Same pairs scored two ways on natural data and official checkpoints; the gate itself with intervals
Two routes with the exact identity; the trained pairing	SCIT swaps values only; residual-stream recomputation papers	The decomposition without recomputation; pairing (1,3), {3}, (1,3), (2,4), (2→3, 4→5), (0,1)(2,3)(4,5) across models
The question register at position 0	—	Transports 0.54 on short chains; costs 59 points when masked
Per-slot faithfulness (alarm confirmation)	Decodability $\neq$ causality in principle (Open Problems, 2025; Decodability, 2026); Ramjee et al. (2026) deploy probes with no causal test	The quantity, measured per slot on latent reasoners; terminal readable 0.42–0.81, faithful 0.000
Sufficiency without necessity; necessity depends on training and scale	Jin et al. (2026); Li et al. (2026) report pieces separately	Swap and mask on the same pairs; CODI-1B transports 0.34 and costs 2.7
The stale copy	—	The unfaithful slot reads the computed value above the carrier
The terminal thought is inert	Partly prior (Cywiński et al., 2025)	Three checkpoints, both routes, monitor-facing
Value-matched repair	Chang et al. (2026): geometric edits, no value-matched donor or controls	Natural failed problems, different-value control, harm, centroid $\approx$ retrieval, K/V route hurts, two value sources, the ceiling
Error detection fails vs confidence	LTO-style latent rewards (LTO, 2026)	The negative on latent reasoners plus placement confirmation
Confidence-gated cascade	FrugalGPT-style cascades; confidence-gated CoT (Lewis-Lim et al., 2510.21007); ThinkRouter (2602.11683) routes between latent and token space by confidence	Released fixed-thought models, no training; the thoughts add nothing to the gate; routing half beats the full switch at 1B/3B
Failure taxonomy; latent-only failures are hard problems	—	Stable across four models, computable at inference
Scale trend 1B→3B→8B; Coconut contrast; ProsQA third profile	Dilgren and Wiegreffe (2026) drop latents on logic tasks	The causal audit at 3B/8B; the twin-pass result on ProsQA

bank row reading the victim's own v^*), the nearest attacker donor, random same-value, centroid, a *random-other* donor reading neither value (the directedness control), the store's K/V, the learned writer, and a donor state written at the readable-but-unfaithful decoy slot. Outcome per victim: *hijacked* (the implied answer), *broken*, *intact*. Defender's side on the hijacked runs: every slot's top-1 reading after the write; the answer's forced-cue probability before and after; out-of-fold ridge detectors on the states, per slot and all slots, folds by victim; and the explicit chain re-evaluated by a calculator against the carrier's reading.

Predictions (H1–H7), written before the run. H1 hijacked ≥ 0.15 at CODI-1B and 8B (≥ 0.10 elsewhere). H2 hijacked $\geq 3\times$ the random-other donor's rate. H3 own-value control intact ≥ 0.90. H4 the decoy slot reads v_{att} on $\geq 50\%$ of attacked victims while $\leq 5\%$ of answers change. H5 among hijacked runs the last slot reads v_{att} on $\geq 50\%$. H6 confidence AUROC ≤ 0.70, carrier-slot detector ≤ 0.70, all-slot detector ≤ 0.80. H7 verification true-positive ≥ 0.80 at clean false-positive ≤ 0.40.

CODI-1B result (first kernel, 394 s on one T4). 416 correctly answered victims; an attacker value and a natural donor exist for 410. Table 7 gives the outcomes.

Table 7: The targeted hijack on CODI-1B: outcome shares over 410 attacked victims. The attacker's value is written at carrier thought 3 unless stated.

Condition	Hijacked	Broken	Intact
Own-value control (a different row reading the victim's own v^*)	0.002	0.061	**0.937**
Nearest attacker donor	**0.210** [0.171, 0.249]	0.173	0.617
Random same-value donor	0.124	0.202	0.673
Centroid of same-value states	0.151	0.188	0.661
Random-other donor (neither value; directedness control)	**0.015**	0.339	0.646
Store K/V only / state + store K/V	0.032 / 0.068	0.120 / 0.154	0.849 / 0.778
Learned writer, absolute / residual	0.102 / 0.159	0.156 / 0.146	0.741 / 0.695
Decoy write at slot 5	0.000	0.000	**1.000**

What held. The attack is real and aimed: one natural vector written into thought 3, with no text produced, moves a fifth of correct answers to the attacker's chosen value and breaks another sixth, against 1.5% for a donor carrying some other value (H1, H2); the own-value control leaves 94%

intact (H3). The decoy is total: the same kind of vector written at the readable-but-unfaithful slot 5 makes a monitor there read the attacker's value on 100% of 296 victims and changes the answer on none (H4). Verification through the model's own explicit chain flags every hijacked run at a 27% false-positive rate on clean ones, or 98% at 15% when comparing answers (H7).

What did not hold, and why it matters more. H5 was wrong: after a successful hijack the downstream slots do *not* copy the attacker's value. Position 4 still reads the original intermediate on 53% of hijacked runs and the attacker's on 13%; slot 5 reads the attacker's on 6%. So after the write the carrier disagrees with its own store. That seam is detectable: an out-of-fold linear detector on slot 4 alone tells hijacked from clean at 0.92 AUROC (0.95 for all written runs), all six slots together 0.92, while the written slot itself is near blind (0.70) and slots 0–2 are at chance (H6b held at the line; H6c was wrong). Confidence also drops, from 0.91 to 0.84 on hijacked runs (AUROC 0.78; H6a wrong). The stale copy that made a monitor *unfaithful* in §5 is, under tampering, *tamper evidence*: the value the carrier handed to the cache survives in the next positions, and a write that changes the carrier without changing them leaves a trace. This is the first positive result for a thought-based monitor in the whole line, and it comes from checking that the slots agree with each other rather than from reading any one of them. SIM-CoT-3B (two carriers) and 8B (no decoy slot; readability tracks faithfulness) follow, to see whether the seam survives scale.

10 Limitations, and what we would do next

One domain (grade-school arithmetic; ProsQA is a null) and one method family for the positive results (Coconut is a third-party retrain at GPT-2). Long-chain pools are small on ASDiv. Outcomes are the first answer token under a forced cue for the twin runs (native decoding for the headline conditions, repair, Coconut and ProsQA). The 8B ran in float16 on two T4s and scored 1.6 points under its paper; some of the 21-point latent–explicit gap may be precision. The cascade's cost is counted in problems routed, not tokens; a token and latency table is the one measurement missing from the efficiency argument. The most significant next step is the causal test of the published safety monitor itself: replicate Ramjee et al. (2026) with their code, train the inert-trigger control their design lacks, and run the audit on the armed state.

References

D. Aswal, T. Palmeira Ferraz, Y. Zhou, and M. Peyrard. Observable patterns are not explanations: A causal-geometric analysis of latent reasoning models. *arXiv:2606.12689*, 2026.

S. Chang et al. Unlocking the black box of latent reasoning: An interpretability-guided approach to intervention. In *ACL*, 2026. arXiv:2606.01243.

B. Cywiński, B. Bussmann, A. Conmy, J. Engels, N. Nanda, and S. Rajamanoharan. Can we interpret latent reasoning using current mechanistic interpretability tools? *Alignment Forum*, December 2025.

J. Dilgren and S. Wiegreffe. Are latent reasoning models easily interpretable? *arXiv:2604.04902*, 2026.

Ding, Huang, and Yang. SCIT. In *EMNLP*, 2026. arXiv:2608.27265.

S. Hao et al. Training large language models to reason in a continuous latent space. *arXiv:2412.06769*, 2024.

Jin, Yang, and Wang. Final checkpoints are not enough: Analyzing latent reasoning faithfulness along training trajectories. *arXiv:2607.06648*, 2026.

Li et al. Dynamics within latent chain-of-thought: An empirical study of causal structure. In *ICML*, 2026. arXiv:2602.08783.

Latent thinking optimization: Your latent reasoning language model secretly encodes reward signals in its latent thoughts. In *ICLR*, 2026. arXiv:2509.26314.

Open problems in mechanistic interpretability. *arXiv:2501.16496*, 2025.

Decodability is not causality: Dissociating probe readouts from behavioral drivers via SAE decomposition. *arXiv:2609.18080*, 2026.

Ramjee et al. Ulterior motives: Detecting misaligned reasoning in continuous thought models. In *ICLR*, 2026. arXiv:2604.23460.

Z. Shen et al. CODI: Compressing chain-of-thought into continuous space via self-distillation. In *EMNLP*, 2025. arXiv:2502.21074.

X. Wei et al. SIM-CoT: Supervised implicit chain-of-thought. In *ICLR*, 2026. arXiv:2509.20317.